Functional Programming in Lean

(The what, why and how of functional programming)

MadLean — 25/03/2026

What is functional programming?

“Functions as first-class citizens”

Functional programming is a programming paradigm where functions are *first-class citizens*, meaning that

- variables may contain functions
- functions may be passed as arguments to other functions
- functions may be returned from other functions

“Functions as first-class citizens”

Most modern programming languages are functional (according to this definition):

Functional programming in Python

```
f = lambda x: x+1
f(3)                                  # ⇝ 4

map(lambda x: x**2, [1,2,3])          # ⇝ <[1,4,9]>
```

“Functions as first-class citizens”

Most modern programming languages are functional (according to this definition):

Functional programming in Python

```
f = lambda x: x+1
f(3)                                # ⇝ 4

map(lambda x: x**2, [1,2,3])        # ⇝ <[1,4,9]>
```

(but does Python count as a functional language??)

Sequence of *instructions* vs. Composition of *expressions*

- Python has some functional *features* but it's still an imperative language: it's based on a series of *instructions* that are *executed*
- Instead, functional languages are based on *expressions* that are *evaluated*

Sequence of *instructions* vs. Composition of *expressions*

- Python has some functional *features* but it's still an imperative language: it's based on a series of *instructions* that are *executed*
- Instead, functional languages are based on *expressions* that are *evaluated*
  - like the λ-calculus!

Sequence of *instructions* vs. Composition of *expressions*

- Python has some functional *features* but it's still an imperative language: it's based on a series of *instructions* that are *executed*
- Instead, functional languages are based on *expressions* that are *evaluated*
  - like the λ-calculus!
  - but also Lisp, OCaml, Haskell, Scala... some of these take the distinction more seriously than others

Why is functional programming better?

Imperative `map`

The `map` applies a function to each value in a list:

`map` in Python

```python
def map(f, l):
    i = 0
    while i < len(l):
        l[i] = f(l[i])
        i += 1
    return l
```

- Step-by-step
- Need to introduce a numeric variable to traverse the list
- Easy to forget e.g. to increment `i`

Functional `map`

The `map` applies a function to each value in a list:

`map` in Lean

```
def map (f : α → β) : List α → List β
| [] ⇒ []
| x::xs ⇒ f x :: map f xs
```

- Abstracts many of the details; more high-level
- If there are two ways to construct lists, we state how to deal with each one
  - Splitting cases
  - Recursion
- Much closer to mathematical reasoning

Global (mutable) state

- In imperative languages, there is a *global state* that can be updated from anywhere in a program

```
count = 0
def g():
    global count
    count += 1
    return count
```

  - The same function called with the same arguments can return different values:

```
>>> g == g
True
>>> g() == g()
False
```

Pure functional languages

- Some functional languages have mutable state, but others (e.g. Haskell or Lean) are "pure": computation is simply expressions being evaluated, *without side effects*
  - any function, when applied to the same arguments, will always return the same value
  - thinking (and proving things) about the code becomes much easier

Pure functional languages

- Some functional languages have mutable state, but others (e.g. Haskell or Lean) are "pure": computation is simply expressions being evaluated, *without side effects*
  - any function, when applied to the same arguments, will always return the same value
  - thinking (and proving things) about the code becomes much easier
- Basically λ-calculus with syntactic sugar

Types in Functional Programming

- We already talked about how the untyped λ-calculus contains terms that "do nothing" when they are evaluated
  - Useless if we want it as a model of computation
- We add types to the λ-calculus to rule out the "bad" terms, so we can work only with the good ones

Types in Functional Programming

- We already talked about how the untyped λ-calculus contains terms that "do nothing" when they are evaluated
  - Useless if we want it as a model of computation
- We add types to the λ-calculus to rule out the "bad" terms, so we can work only with the good ones
- But what types should we choose?

Simply-Typed λ-Calculus (STLC)

The Simply-Typed λ-Calculus ($\lambda\rightarrow$) has

- whatever base types we choose (e.g. `Nat`, `Bool`, `String`)
- function types ($\tau_1 \rightarrow \tau_2$)

Simply-Typed λ-Calculus (STLC)

The Simply-Typed λ-Calculus ($\lambda\rightarrow$) has

- whatever base types we choose (e.g. `Nat`, `Bool`, `String`)
- function types ($\tau_1 \rightarrow \tau_2$)

Many functions (e.g. the identity $\lambda x.x$, function application $\lambda x.\lambda y.xy$...) work the same on values of any type; there's no need to reimplement one version for each type.

Polymorphic λ-Calculus (System F)

The Polymorphic λ-Calculus or Second-Order λ-Calculus ($\lambda 2$) fixes this by introducing type variables (variables ranging over types). We can write for example:

(Parametric) Polymorphism in Lean

```
def id (x : α) : α := x
def apply (f : α → β) (x : α) : β := f x
```

- (this is different from the polymorphism in Python or Java; more on this later)

Polymorphic λ-Calculus (System F)

The Polymorphic λ-Calculus or Second-Order λ-Calculus ($\lambda 2$) fixes this by introducing type variables (variables ranging over types). We can write for example:

(Parametric) Polymorphism in Lean

```
def id (x : α) : α := x
def apply (f : α → β) (x : α) : β := f x
```

- (this is different from the polymorphism in Python or Java; more on this later)
- But there's not much we can do if the only "generic" types are plain variables

λ-Calculus with Type Operators (System Fω)

The λ-Calculus with Type Operators ($\lambda\underline{\omega}$) introduces a new way to define types from simpler types. For example:

```
inductive List (α : Type) : Type where
| nil : List α
| cons : α → List α → List α

inductive Sum (α : Type) (β : Type) : Type where
| inl : α → Sum α β
| inr : β → Sum α β
```

`List` and `Sum` are "type operators" because they act on types to produce other types.

- $\lambda\underline{\omega}$ is not very useful on its own, so it's commonly used together with $\lambda 2$. This combination is System Fω (or $\lambda\omega$).

Dependent λ-Calculus (λP)

The Dependent λ-Calculus (λP) allows types to depend on values:

```
inductive Vec (a : Type) : Nat → Type where
| nil : Vec a 0
| cons : a -> Vec a n -> Vec a (n+1)

#check Vec Int                    -- Vec Int : Nat → Type
```

Computationally this adds nothing; it's only useful to have more expressive types (i.e. to reject more programs).

Lambda cube (Barendregt)

$\lambda\!\rightarrow$

$\lambda\!\rightarrow$ terms depend on terms (functions)

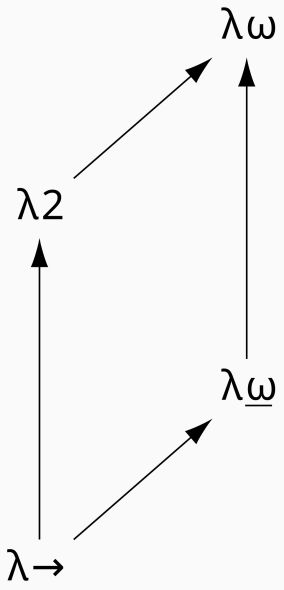
Lambda cube (Barendregt)

$$\begin{tikzcd} \lambda 2 \\ \lambda\!\to \arrow[u] \end{tikzcd}$$

$\lambda\!\to$ terms depend on terms (functions)

$\lambda 2$ terms depend on types (think $id_{\texttt{Nat}}, id_{\texttt{Bool}}, \ldots$)

Lambda cube (Barendregt)

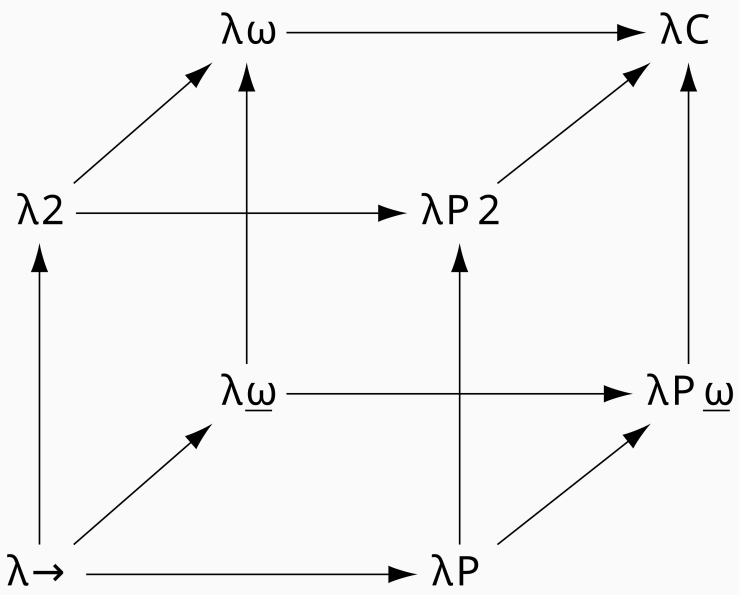


$\lambda\!\rightarrow$ terms depend on terms (functions)

$\lambda 2$ terms depend on types (think $id_{\texttt{Nat}}, id_{\texttt{Bool}}, \ldots$)

$\lambda\underline{\omega}$ types depend on types (type constructors)

Lambda cube (Barendregt)



$\lambda\!\rightarrow$ terms depend on terms (functions)

$\lambda 2$ terms depend on types (think $id_{\texttt{Nat}}, id_{\texttt{Bool}}, \ldots$)

$\lambda\underline{\omega}$ types depend on types (type constructors)

λP types depend on terms

How to do stuff with functional programming

Ad-hoc Polymorphism

- General interface that can be implemented by different types in different ways (Java interfaces or abstract classes)
- In Lean, this is done through type classes:
  - classes define the abstract interface through generic methods that should be implemented
  - to define a particular instance, we implement those methods for our particular context

```
class Mul (a : Type) where
mul : a → a → a

class Semigroup (G : Type) extends Mul G where
protected mul_assoc : ∀ a b c : G, (a*b)*c = a*(b*c)
```

In contrast, *parametric* polymorphic functions do the same operations, uniformly, on every type

Example: Functors

- Type constructor `F : Type -> Type`
- Lift a function `f : α → β` to a function `f' : F α → F β`

Example: Functors

- Type constructor `F : Type -> Type`
- Lift a function `f : α → β` to a function `f' : F α → F β`

```
class Functor (F : Type → Type) where
map : (α → β) -> (F α -> F β)
```

Implementations are expected to satisfy

- `map id = id`
- `map (f ∘ g) = map f ∘ map g`

but it's not required to prove it. The class `LawfulFunctor` states these theorems precisely

Side effects

Programs need to interact with the outside world at some point:

```
def greet_user():
    name = input("What's your name? ")
    return "Hello, " + name
```

...but this can't be *pure*: each time you call this function you may get a different value

Side effects

In pure functional languages, side effects are faked by wrapping the value in a datatype:

```
def greetUser : IO String := do
let name ← (← IO.getStdin).getLine
return "Hello, ".append name
```

- Doesn't return a `String`, but a `IO String`
  - Think of `IO α` as an envelope around values of type `α`

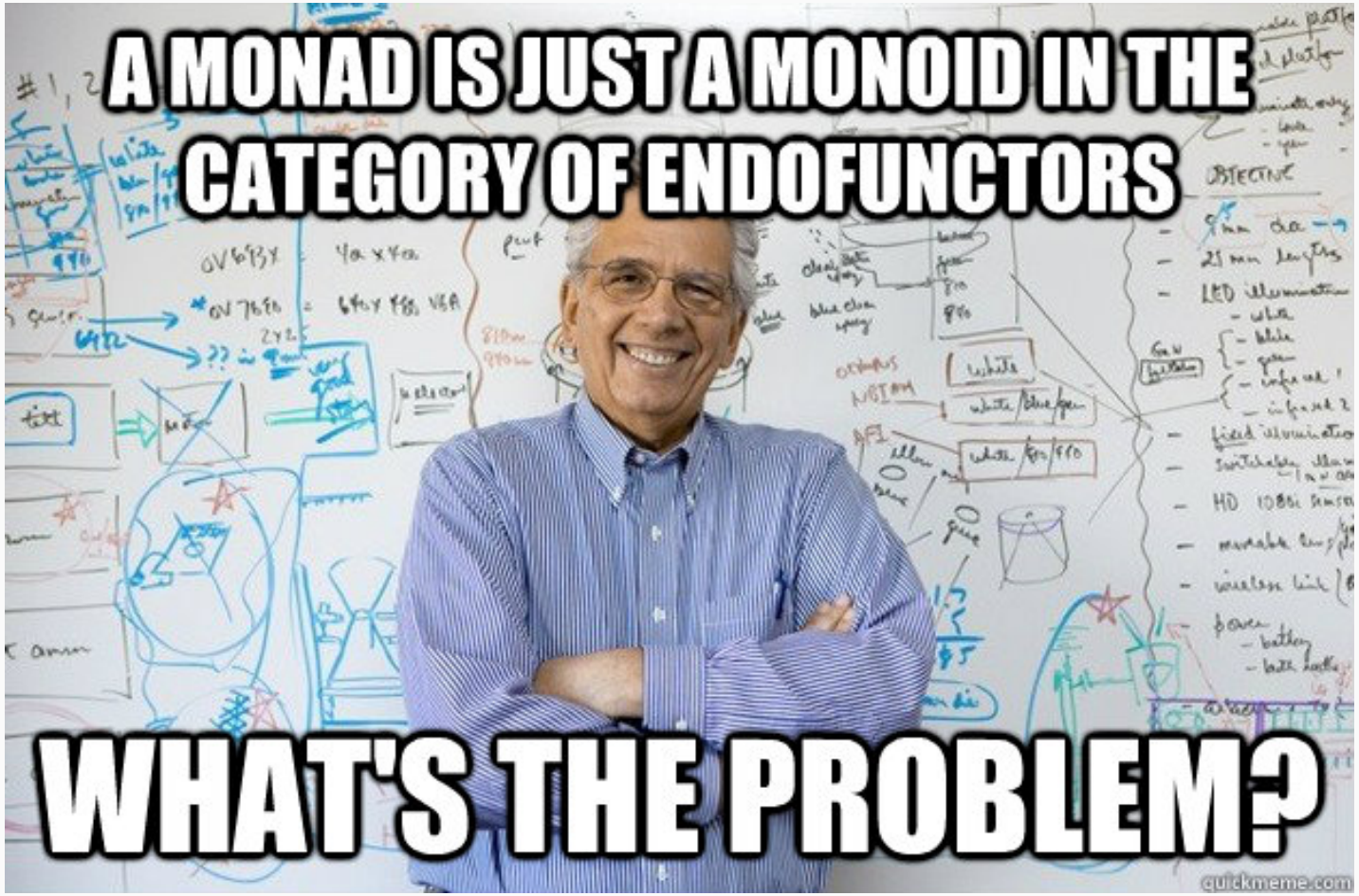
Side effects

In pure functional languages, side effects are faked by wrapping the value in a datatype:

```
def greetUser : IO String := do
let name ← (← IO.getStdin).getLine
return "Hello, ".append name
```

- Doesn't return a `String`, but a `IO String`
  - Think of `IO α` as an envelope around values of type `α`
  - `greetUser` is pure because it always returns the same envelope; the values inside may change

Monads


A MONAD IS JUST A MONOID IN THE CATEGORY OF ENDOFUNCTORS
WHAT'S THE PROBLEM?
quickmeme.com

Monads

A type constructor `M : Type → Type` is a monad if (it's a functor and) there are two natural transformations

1. $\eta : 1 \Rightarrow M$
2. $\mu : M \circ M \Rightarrow M$

satisfying the monoid axioms.

Monads

A type constructor `M : Type → Type` is a monad if (it's a functor and) there are two families of functions

1. $\eta_\alpha : \alpha \to M\alpha$
2. $\mu_\alpha : M(M\alpha) \to M\alpha$

satisfying the monoid axioms.

"Purity" is preserved because there is no function $M\alpha \to \alpha$: once you enter the monad, there is no way out

- (i.e. no way to do it uniformly, for an arbitrary monad)

Monads in Lean

Monads are a type class extending `Applicative` (= `Functor` + some other things) and `Bind`. The main methods for us are

1. `pure : α → m α` (also called `return`)
2. `bind : m α → (α → m β) → m β`
   - combines `map` and $\mu : (\alpha \to M\beta, M\alpha) \xrightarrow{\texttt{map}} M(M\beta) \xrightarrow{\mu_\beta} M\beta$
   - thanks to μ, we can keep working inside the monad and not end up inside a tower $M(M(M\ldots)))$

Many common data structures are monads

Example: `Option`

The `Option` monad models computations that may fail:

```
inductive Option (α : Type) : Type where
| none : Option α
| some : α → Option α
```

Let's use it to validate a record

```
structure Person where
  name : String
  id : Nat
deriving Repr
```

Example: Option

```
def validateName  (s : String) : Option String :=
  if s.length > 0 then some s else none
def validateId    (i : Nat)    : Option Nat    :=
  some i -- always OK

def validate (name : String) (id : Nat) :
    Option Person := do
  let name' ← validateName name
  let id' ← validateId id
  return ⟨name', id'⟩
```

```
#eval validate ""     12345678  -- none
#eval validate "Luis" 12345678  -- some { .. }
```

Example: `List`

The `List` monad models computations that can have more than one result (or no results at all).

For some reason `List` is not an instance of `Monad` by default in Lean4. Let's write our own!

Example: List

The `List` monad models computations that can have more than one result (or no results at all).

```
instance : Pure List where
pure x := [x]

def List.bind (xs : List α) (f : α -> List β) :
   List β := match xs with
| [] ⇒ []
| x::xs' ⇒ List.append (f x) (List.bind xs' f)

instance : Bind List where
bind := List.bind

instance : Monad List where
```

Example: List

Non-negative numbers have two square roots, and negative numbers don't have any:

```
def sqrts (x : Float) : List Float :=
  if x ≥ 0
  then let y := Float.sqrt x; [y, -y]
  else []
```

An equation like $ax^2 + bx + c = 0$ has one solution for each square root of $b^2 - 4ac$:

```
def solve (a b c : Float) :
    List Float := do
  let Δ ← sqrts (b^2 - 4*a*c)
  return (-b + Δ)/(2*a)
```

Example: List

```
#eval solve (1) ( 0) (-1)
-- x² - 1 → x=1, x=-1
#eval solve (1) (-2) ( 1)
-- x² - 2x + 1 → x=1 twice
#eval solve (2) (-2) (-12)
-- 2x² - 2x - 12 → x=3, x=-2
#eval solve (1) ( 0) ( 1)
-- x² + 1 → no solution
```

References

- https://lean-lang.org/functional_programming_in_lean/
- https://lean-lang.org/doc/reference/4.26.0/Functors___-Monads-and--do--Notation/